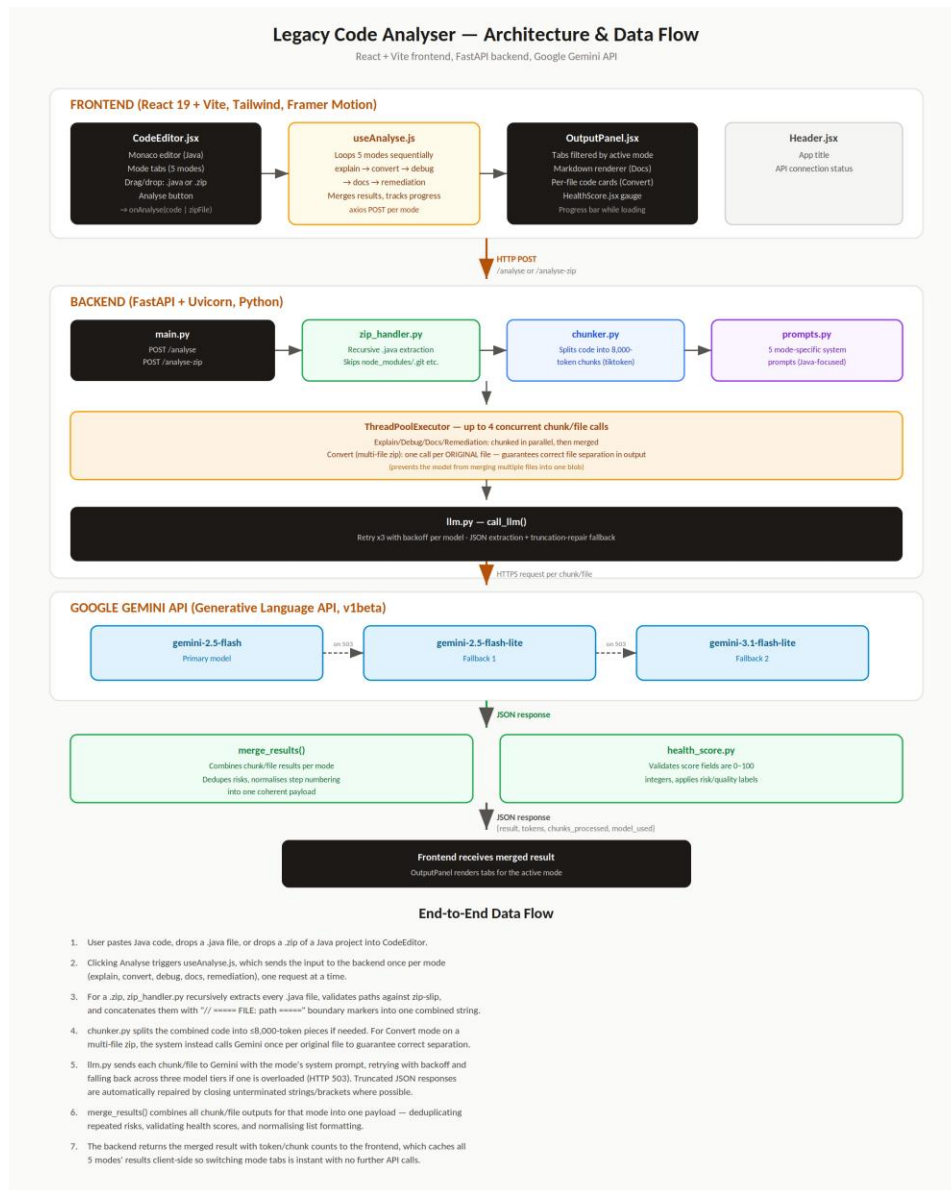


Legacy Code Analyser

Architecture, Data Flow & Future Roadmap

1. Architecture & Data Flow Diagram

The diagram below shows the three layers of the system — the React frontend, the FastAPI backend pipeline, and the Google Gemini API with its fallback chain — along with the end-to-end data flow for a single Analyse click.



2. Future Roadmap

The roadmap below is organised into three horizons: near-term hardening of what already exists, mid-term feature expansion, and longer-term platform features. Effort estimates assume the current two-person-plus engineering pace seen during the hackathon build.

Horizon 1 — Near Term (Hardening)

Item	Why	Rough Effort
Persist chat/analysis history server-side	Currently all state lives in the browser tab; refreshing the page loses results.	2-3 days
Add automated tests for chunker, merge_results, zip_handler	These three modules carry the most logic risk (chunk boundaries, merge correctness, zip-slip safety) and have no test coverage yet.	3-4 days
Structured logging instead of print() debug statements	Current DEBUG RESULT print statements work for development but won't scale to diagnosing production issues.	1-2 days
Configurable concurrency and chunk size	MAX_CONCURRENT_CHUNKS and the 8,000-token chunk size are hardcoded; making them env-configurable allows tuning per deployment without code changes.	1 day

Horizon 2 — Mid Term (Feature Expansion)

Item	Why	Rough Effort
Conversational follow-up chat	Right now each analysis is a one-shot result; a chat interface would let a user ask follow-up questions like “explain the bug in TransactionProcessor.processTransfer in more depth” against the already-analysed code, without re-running all 5 modes. See Section 3 for a feasibility assessment.	1-2 weeks
Support additional JVM languages (Kotlin, Scala)	Many legacy Java shops have mixed JVM codebases; broadening zip_handler's extension list and adjusting prompts would extend reach without a full rebuild.	3-5 days
Diff-aware re-analysis	Re-running all 5 modes on a slightly modified file repeats work; caching by file hash and only re-analysing changed files would speed up iterative use.	1 week
Exportable PDF/Word report of full analysis	Useful for sharing findings with stakeholders who won't use the web app directly; the existing docx-generation approach used for this very document could be templated.	3-4 days

Horizon 3 — Longer Term (Platform)

- Multi-user accounts and saved project history, so a team can revisit past analyses rather than re-uploading each session.
- CI/CD integration (e.g. a GitHub Action) that runs Find Bugs and Health Score automatically on pull requests touching legacy modules.
- Pluggable LLM backend so teams can point the tool at their own model deployment (Azure OpenAI, Bedrock, on-prem) instead of only Gemini.
- Confidence/verification layer that cross-checks AI claims (e.g. method names mentioned in documentation) against a real Java AST parse, to reduce hallucination risk discussed in Section 3.

3. Feasibility Brief: Conversational Chat Feature

This section assesses adding a chat interface so users can ask follow-up questions about an already-analysed codebase, rather than only receiving the fixed 5-mode output.

What it would look like

After an Analyse run completes, a chat panel (or a 6th tab) would let the user type a question — e.g. “why is the SQL injection risk in DatabaseManager rated high?” or “show me a fixed version of just the transfer method.” The assistant would answer using the original code plus the already-generated analysis (Explain/Debug/Docs/Remediation results) as context, rather than re-analysing from scratch.

Technical Feasibility — High

This is the most natural extension of the current architecture, for several concrete reasons:

- The backend already has a working `call_llm()` function in `llm.py` with retry/fallback and JSON-repair logic; a chat endpoint would reuse it directly, swapping the JSON-only system prompt for a conversational one.
- The original combined code string (with file boundary markers) and the 5 modes' results are both already held in memory per analysis — they're exactly the context a chat turn needs.
- FastAPI supports streaming responses natively, which would let chat replies appear token-by-token rather than waiting for a full response, improving perceived speed over the current single-shot UX.
- The frontend already has React state management (`useAnalyse.js`) and a tabbed panel pattern (`OutputPanel.jsx`) that a chat tab would slot into with moderate, not extensive, rework.

Key Risks & Open Questions

Risk	Mitigation
Token cost grows with conversation length	Each chat turn would need to resend relevant context (code + prior analysis) since Gemini has no server-side conversation memory across separate API calls; for a multi-file zip this could get expensive. Mitigate by sending only the

Risk	Mitigation
	original analysis summary plus the specific file(s) referenced, not the full codebase, on every turn.
Hallucination risk is more visible in open-ended Q&A than in structured JSON output	As discussed in earlier project conversations, temperature and JSON mode don't prevent the model from confidently misdescribing code. A chat answering arbitrary questions has more surface area for this than the current fixed-schema modes. Mitigate with the same “ground claims in real method/variable names” prompting already used elsewhere in this project, and consider a future AST cross-check (see Horizon 3).
Multi-turn state management	The current frontend holds one merged result per analysis run with no chat history concept. A new piece of state (chat message list) and a new backend endpoint maintaining conversation context per session would need to be added — moderate, well-scoped work given the existing patterns.
Rate limiting under interactive use	Gemini's free-tier rate limits, already a recurring issue in this project (503 errors), would be hit more easily by a chat that invites many small follow-up requests rather than one batched analysis. The existing retry/fallback chain in llm.py would need to extend to chat calls too.

Recommendation

Technically feasible with moderate effort (estimated 1-2 weeks for a working version), since it builds directly on infrastructure that already exists rather than requiring a new subsystem. The main design decision is scoping how much context to resend per turn to balance cost against answer quality. Recommended as a Horizon 2 item: valuable, buildable on the current foundation, but best sequenced after the Horizon 1 hardening items (particularly structured logging and test coverage) so a new conversational surface doesn't compound existing reliability gaps.